

Digital Public Transport Modernisation Programme

Technical Proposal for the Addis Ababa City Administration Transport Bureau

Working title of the platform: Addis One

Field	Detail
Submitted to	Addis Ababa City Administration — Transport Bureau (also styled "Road and Transport Bureau")
Submitted by	Addis One Technical Team
Date	October 2026
Document type	Technical proposal, request for collaboration and authorisation
Version	1.0 — For Bureau review
Classification	Commercial-in-confidence until reviewed
Basis	Inspected from the built system; all technical claims trace to running code

How to read this document

This proposal is written from an **inspection of a working system**, not from a conceptual design. Every technical statement in Parts III–V describes software that exists, runs, and is covered by automated tests. Where a capability is *not yet built*, this document says so plainly rather than implying otherwise — see **Part IV, Section 12: Honest Capability Statement**.

Sections are written to be lifted out independently, so the Bureau can circulate Part VII (costs) to finance, Part IV (technical procedures) to IT, and Part IX (governance) to legal review without reassembly.

Contents

Part I — Executive Summary

- 1.1 The request in one paragraph
- 1.2 Why this matters now
- 1.3 What Addis One is
- 1.4 The five guarantees the system enforces
- 1.5 Evidence of maturity
- 1.6 What we are asking for
- 1.7 Investment and return, in brief

Part II — Problem Statement and Objectives

- 2.1 The operational problem
- 2.2 Objectives
- 2.3 Principles
- 2.4 Scope

Part III — System Architecture

- 3.1 The governing architectural rule
- 3.2 Layer view
- 3.3 Deployment shape: modular monolith
- 3.4 Module inventory
- 3.5 Technology stack
- 3.6 Realised reference network
- 3.7 Cryptographic credential design
- 3.8 How invariants I1 and I2 are enforced in code
- 3.9 Authorisation model
- 3.10 Data model summary

Part IV — Technical Procedures

- 4.1 Procedure 1 — Fare rule drafting, approval and activation
- 4.2 Procedure 2 — Fare calculation
- 4.3 Procedure 3 — Journey planning
- 4.4 Procedure 4 — Purchase, payment and ticket issuance
- 4.5 Procedure 5 — Ticket validation (boarding inspection)
- 4.6 Procedure 6 — Shift and cash reconciliation
- 4.7 Procedure 7 — Complaints and service assurance
- 4.8 Procedure 8 — Identity, authentication and session security
- 4.9 Procedure 9 — Governance reporting
- 4.10 Procedure 10 — Audit and tamper evidence
- 4.11 API reference summary
- 4.12 Honest capability statement

Part V — Governance, Security and Data Protection

- 5.1 Security architecture
- 5.2 Data protection and privacy
- 5.3 Sovereignty and lock-in

Part VI — Implementation Roadmap

- 6.1 Phasing philosophy
- 6.2 Phase 0 — Foundation (months 1–2)
- 6.3 Phase 1 — Pilot (months 3–10)

6.4 Phase 2 — Expansion (months 11–18)

6.5 Phase 3 — Integration (months 19–26)

6.6 Phase 4 — Citywide (months 27–50)

Part VII — Cost, Resources and Capacity

7.1 Phase 0 + Phase 1 (pilot, 10 months)

7.2 Phases 2–4 (citywide, 5 years)

7.3 Return model — and its honest limits

7.4 Team and capacity

7.5 Training and change management

Part VIII — Success Metrics

8.1 Revenue integrity (01, 02)

8.2 Compliance and enforcement (03)

8.3 Service quality (04)

8.4 Data quality (05)

8.5 Adoption and equity

8.6 System health

Part IX — Risk Register

9.1 Risks to this proposal specifically

Part X — The Request

10.1 What we ask the Bureau to approve

10.2 What we commit to

10.3 Immediate next steps

10.4 Contact

Appendix A — Glossary

Appendix B — Architecture Decision Records

Appendix C — Technical evidence available for Bureau review

Part I — Executive Summary

1.1 The request in one paragraph

We ask the Transport Bureau to authorise a **pilot** of **Addis One**, a digital public transport operating, ticketing and assurance platform, on a defined set of Anbessa and private-operator routes; and to establish a joint Bureau–Team programme office to govern its delivery. We are not asking the Bureau to adopt an off-the-shelf product. We are asking it to co-own a system whose revenue-integrity and audit guarantees are enforced structurally in code, so that the Bureau can verify them rather than trust them.

1.2 Why this matters now

The Bureau's own 2011 Transport Policy of Addis Ababa set out objectives that remain unmet two decades on: enhancing transport service provision, employing integrated and modern traffic management, and strengthening financial capacity. Service provision remains fragmented, fare determination is opaque, and the Bureau has limited ability to see its own network in real time.

The Bureau is not starting from zero. The Addis Ababa City Roads Authority procured a 644 million Birr Intelligent Transport System for Anbessa's fleet — vehicle tracking, electronic fare collection, passenger information and CCTV — delivered by Qingdao Hisense Trans Tech and China Shandong International, with a two-year installation and six-year maintenance term. **That project answers "where are the buses." It does not answer the questions the Bureau actually administers: how many passengers, who paid the correct fare, who validated which ticket, and where the revenue went.**

This proposal addresses that gap specifically. It does not replace the ITS investment; it consumes it.

1.3 What Addis One is

A citywide **transport operating and assurance platform**. Not a ticketing app. Ticketing is one capability of a system whose primary purpose is to give the Bureau a defensible, real-time, auditable view of the public transport network.

Capability	What it delivers to the Bureau
Journey planning	Multi-leg routing across modes with fares computed by published, versioned policy
Versioned fare engine	Every fare reproducible and explainable years later; fare changes dual-control approved
Digital ticketing	Server-signed Ed25519 QR credentials verifiable offline by inspector handhelds
Payment orchestration	Telebirr, CBE Birr, bank, USSD, wallet and cash/agent through one pipeline
Cash reconciliation	Shift-level float, declared cash and variance — the missing control on cash revenue
Live assurance	Complaints, overcharge, safety and lost-property with geospatial capture
Governance dashboard	Revenue, operations, network, fare and fleet reporting, role-scoped
Tamper-evident audit	Append-only, hash-chained log of every privileged action

1.4 The five guarantees the system enforces

These are the load-bearing claims. Each is enforced at the **database or cryptographic** layer, not by application convention, because application-layer checks are bypassable and this is a revenue system.

#	Guarantee	Enforcement mechanism
I1	A ticket exists only against a confirmed payment	State machine has exactly one edge into <code>TICKET_ISSUED</code> , from <code>PAYMENT_CONFIRMED</code>
I2	Idempotency is a database constraint	<code>payments.idempotencyKey</code> is <code>UNIQUE</code> ; double-tap cannot double-charge
I3	Fares are versioned and tickets pin their version	<code>Ticket.fareRuleVersion</code> NOT NULL; history is never rewritten
I4	QR payloads carry no commercial data	Credential holds identifiers, timestamps, nonce, signature only
I5	Every privileged action is audited	Append-only <code>AuditEvent</code> with a chained hash

Detail matters on I1. The common catastrophic bug in mobile ticketing is treating "the passenger came back from the payment redirect" as proof of payment. This system structurally cannot make that error: the payment orchestrator has no method that mints a ticket. A redirect return is `REQUIRES_ACTION`, which is explicitly *not* confirmation, and a test asserts exactly that.

1.5 Evidence of maturity

This is not a prototype. As built and verified at the time of submission:

- **~9,400 lines** of TypeScript across 58 source files
- **141 automated tests** in 7 suites, all passing, covering the fare engine, ticket state machine, payment orchestrator, credential signer, authorisation policy and dashboard query layer
- **~40 database models** with invariants expressed as schema constraints
- **12 staff roles** with a single centralised authorisation policy file
- **Two Flutter applications** (passenger, staff/inspector) and a web operations dashboard
- **Real Addis Ababa network data** seeded: 15 stops with Amharic names and true coordinates, 6 routes, zone-based fare rules with a superseded v1 and active v2

The fare rule seeding deliberately ships `CITY-BUS v1` as `SUPERSEDED` and `v2` as `ACTIVE` so that invariant I3 is *testable*: a ticket issued before the change pins v1, and the same route today quotes v2 — so the historical charge stays explicable. That is the level of intent applied throughout.

1.6 What we are asking for

1. **A memorandum of understanding** establishing a joint Bureau–Team programme office.
2. **Authorisation for a pilot** on an agreed route corridor, recommended as 6–10 routes serving the central corridor (Piazza–Derg Square–Merkato–Bole–Kality), where the seeded reference data already exists and demand density is highest.
3. **Data and regulatory access** — network reference data, fare policy documents, operator agreements, and the ITS feed.
4. **A joint assurance mandate** — Bureau auditors with read access to the audit chain and revenue reporting.

We ask for no exclusivity and no procurement commitment in this document. We ask for a pilot that produces evidence the Bureau can use to make a later, informed decision.

1.7 Investment and return, in brief

Indicative figures, detailed in Part VII:

- **Pilot (18 months):** approximately **ETB 34–52 million** total, of which the Bureau's cash requirement is a minority share — the remainder being Team investment, in-kind operator contribution and device financing.
- **Citywide (Phases 2–4, 5 years):** approximately **ETB 380–620 million**.

Return rests on three measurable sources: recovered revenue leakage from cash handling, reduced fare disputes resolved from evidence rather than argument, and enforcement efficiency from knowing who boarded without paying.

We deliberately do not promise a percentage. Any revenue figure before a pilot would be invented, and the Bureau should treat our projections as estimates to be validated by measurement — which is precisely what the pilot is for.

Part II — Problem Statement and Objectives

2.1 The operational problem

Addis Ababa's public transport is delivered by a mix of state and private operators running buses, minibuses, taxis and, increasingly, ride-hailing. The Bureau regulates this network but cannot currently see it as a single system. Five specific failures follow from that.

2.1.1 Fare determination is invisible and unenforceable

Fares are set by policy but applied by convention at the point of boarding. A conductor's assertion of the correct fare is difficult to challenge without evidence. There is no authoritative record of what fare *should* have been charged for a specific origin, destination, mode and time — so disputes are resolved by argument, not by record.

Consequence: passengers overpay without recourse; operators face unfounded disputes; the Bureau cannot measure compliance with its own fare policy.

2.1.2 Cash revenue is untracked from the vehicle to the treasury

Cash is the dominant payment method for public transport. It changes hands at the door and is collected by staff. There is typically no shift-level record of what float a vehicle started with, what was sold, and what was declared at the end of a shift.

Consequence: shortfalls are discovered, if at all, only through suspicion or after-the-fact audit. There is no control environment to audit *against*.

2.1.3 Fare evasion is unobservable

Whether a fare-paying passenger actually travelled is not recorded anywhere. A ticket-free system means boarding is unenforced; a printed-ticket system creates its own counterfeit problem.

Consequence: the Bureau cannot size the revenue gap, so it cannot target enforcement or estimate the return from digitisation.

2.1.4 Service quality is reported anecdotally

Overcharging, safety incidents, discrimination and lost property reach the Bureau through complaints that are unstructured, unlocated and unmeasured. There is no category taxonomy, no geospatial capture, no resolution workflow and no trend analysis.

Consequence: recurring problems on specific routes, operators or stops stay invisible, and the Bureau cannot direct its limited enforcement resource.

2.1.5 Network reference data is not authoritative in software

Routes, stops, vehicles, operators and fares exist as paperwork and in operator heads. Planning, dispatch and compliance work cannot be automated because the data is not in a form software can use.

Consequence: every new initiative re-keys the same network data, and none of it is reusable.

2.2 Objectives

The programme is designed against five objectives, each mapped to a measurable success criterion in Part VIII.

#	Objective	Success criterion (pilot)
01	Make fare policy computable and explainable	100% of issued tickets carry a resolvable fare rule version; any fare reproducible from stored inputs
02	Bring cash revenue under control	≥98% of staff shifts closed with a declared cash figure; variance trend measured and falling
03	Make fare evasion observable	Validation rate and outcome distribution measurable per route/operator/day
04	Turn complaints into structured assurance data	100% of complaints categorised, geolocated and tracked to resolution
05	Establish authoritative network reference data	Bureau-approved stop, route, vehicle and operator registers loaded and version-controlled

2.3 Principles

These constrain every design decision, and are stated here because the Bureau should be able to hold us to them.

1. **The server is authoritative; the phone is not.** A fare is not correct because a phone said so. Every authoritative decision — fare calculation, ticket issuance, signature, validation outcome — is made server-side.
2. **Fail closed, visibly.** Where the fare engine matches no rule, it does not invent a price. It applies a flagged fallback and raises an alert, so the anomaly is a monitoring event rather than a silent wrong charge.
3. **Every privileged action is attributable.** Actor, device, IP, before/after value and reason are recorded, and the record is tamper-evident.
4. **Cash is a first-class payment method, not an exception.** Agent sales flow through the same orchestrator as mobile payments, producing the same auditable records.
5. **Privacy by construction.** A photo of a passenger's QR reveals nothing about them. Live position history is limited to operational need.
6. **Explainability over cleverness.** The journey planner uses a simple, explainable algorithm. When a passenger disputes a fare, the Bureau can show *why* the option existed. An opaque algorithm cannot be explained at all.
7. **Open standards, sovereign operation.** The Bureau owns its data, its keys and its deployment. No vendor lock-in, no proprietary wire format.

2.4 Scope

In scope for the pilot

- Bus mode on an agreed central corridor (taxi and rail data models are present; taxi and rail are Phase 2)
- Digital ticketing with offline-capable validation
- Telebirr and CBE Birr payment integration, plus cash and agent sales
- Fare policy administration with dual-control approval
- Staff shift and cash reconciliation
- Complaints and assurance workflow
- Governance dashboard with role-scoped reporting
- Staff, driver and inspector mobile applications

Explicitly out of scope for the pilot

- Road traffic management, signal control and enforcement (Bureau/AACRA domain)
- Physical infrastructure and stop construction

- Railway ticketing automation
 - Taxi and ride-hailing fare integration
 - Vehicle maintenance and workshop systems
 - Rewriting or replacing the AACRA ITS investment
-

Part III — System Architecture

This part describes the architecture as implemented. It is the technical basis for Part IV's operating procedures.

3.1 The governing architectural rule

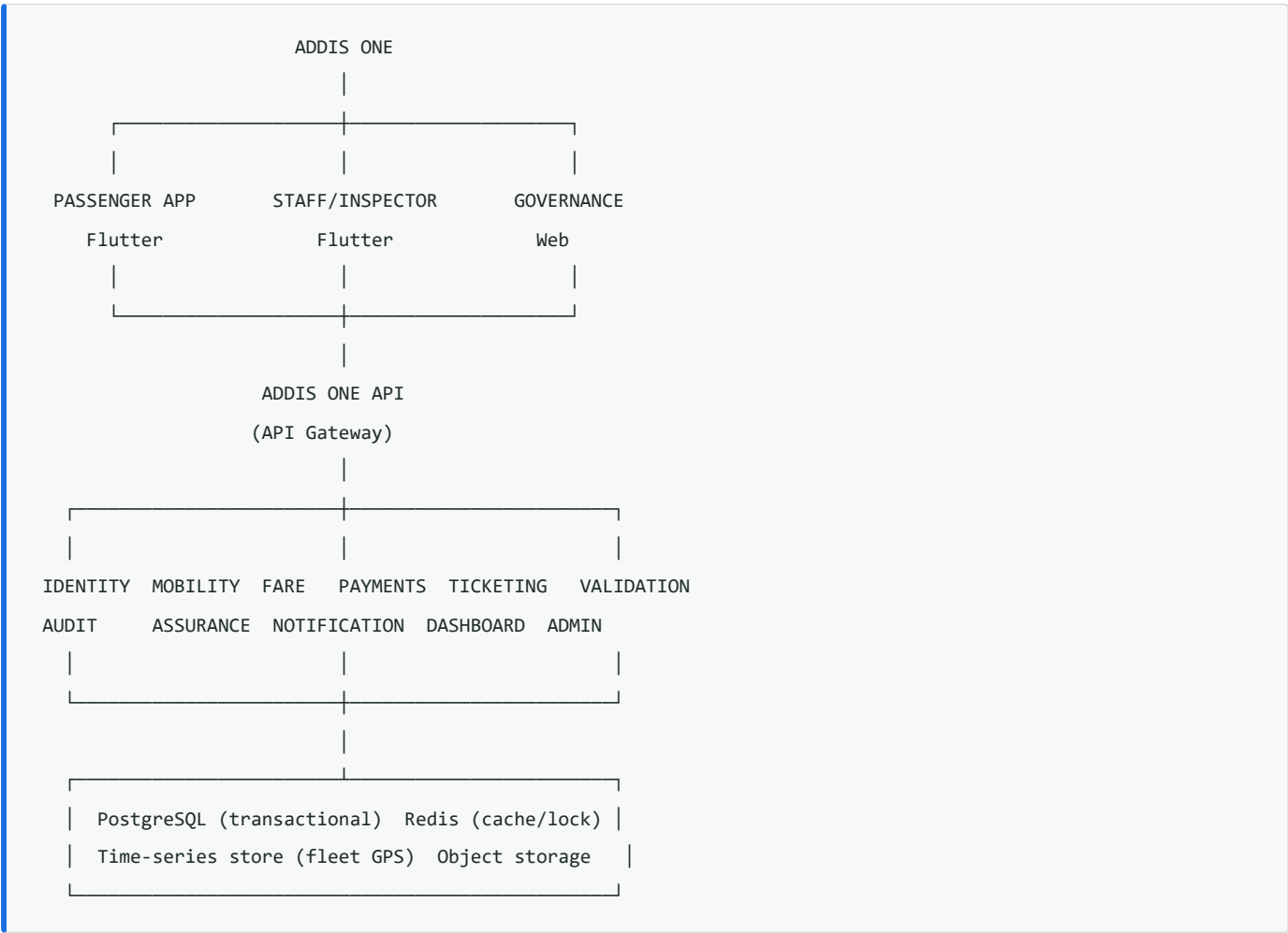
The mobile application is the client layer. It is not the system.

A ticket is not valid because a phone said so. A fare is not 25 Birr because a conditional in Dart said so. A payment is not successful because the passenger returned from a redirect screen.

Every authoritative decision — fare calculation, ticket issuance, QR signature, validation outcome — happens on the server. The client renders server truth.

The alternative fails catastrophically. A modified client could mint valid tickets and forge payment confirmations. For a public transport revenue system that is a revenue-integrity failure, not a bug.

3.2 Layer view



3.3 Deployment shape: modular monolith

The backend is a **single deployable NestJS application** organised into modules with explicit boundaries and no cross-module table writes. Modules communicate through exported services.

This is a deliberate decision, recorded as ADR 0001. The rationale matters for the Bureau because it determines when cost and complexity rise:

- A ticket and its confirming payment must commit **atomically**. Splitting these into separate network services on day one introduces distributed transactions and partial-failure modes into a system whose correctness depends on exactly that guarantee.
- Modules are shaped so that later extraction into separate services is a **deployment change, not a rewrite**. If fleet-GPS ingestion needs independent scaling — and it will — it is already carved out.
- One deployment, one migration history, straightforward local development and handover to Bureau staff.

The cost accepted: a single scaling unit, and module boundaries enforced by discipline and review rather than by infrastructure. Mitigated by a single centralised authorisation policy file and code review at the boundary.

3.4 Module inventory

Module	Responsibility	Key artefacts
Auth / Identity	OTP sign-in, JWT issuance, staff sessions, device enrolment	<code>auth.controller</code> , <code>staff.guard</code> , <code>policy</code> , <code>token.service</code>
Journeys	Journey planning, stop search, nearest-stop	<code>journey-planner.service</code>
Mobility	Driver trips, manifests, trip lifecycle	<code>mobility.controller</code> , <code>mobility.service</code>
Fares	Pure fare calculation domain logic	<code>fare-engine.ts</code>
Payments	Orchestration, provider adapters, idempotency	<code>payment-orchestrator</code> , <code>payments.controller</code>
Ticketing	Issuance, QR credential signing, state machine	<code>ticket-issuance.service</code> , <code>credential-signer</code> , <code>ticket-state-machine</code>
Validation	Inspector scanning, offline reconciliation	<code>validation.service</code>
Admin	Network reference data, fare rule authoring, staff	<code>admin.controller</code> , <code>admin.service</code>
Assurance	Complaints workflow	<code>assurance.controller</code>
Dashboard	Read-only governance reporting	<code>dashboard.controller</code> , <code>database-inspector.service</code>
Audit	Append-only hash-chained event log	<code>AuditEvent</code> model

3.4.1 Why the dashboard is read-only by construction

The dashboard controller exposes **no** POST, PUT or DELETE handlers. It is a reporting surface over records that are already immutable or governed elsewhere. A dashboard that could approve a fare or void a ticket would need dual-control audit on every action — and that belongs in the admin module, not bolted onto a screen whose guards were only ever reasoned about for reads.

3.4.2 Why ticketing and validation are separate modules

Ticketing **issues and signs**. Validation **verifies and records**. The scanner never mints a ticket; the issuer never validates one. Separating them means neither surface can drift into the other's authority by accident.

3.5 Technology stack

Layer	Technology	Rationale
API	NestJS 10, TypeScript 5.7, Node ≥20	Structured modules, first-class DI, guards match the authorisation model

Layer	Technology	Rationale
Persistence	PostgreSQL 16 via Prisma 5	Transactions are the backbone of invariants I1 and I2
Cache / lock	Redis 7	Rate limiting, idempotency short-circuit, OTP challenge cache
Time-series	TimescaleDB (Postgres 16)	Fleet GPS, deliberately outside the transactional database
Clients	Flutter (Dart), Android-first	One codebase, low-end device support, offline capability
Dashboard	Server-served static web	Officers always see the deployed version, never a stale cached copy
Cryptography	Ed25519 via Node <code>crypto</code>	See §3.7
Documentation	OpenAPI via <code>@nestjs/swagger</code>	Self-describing API; every route discoverable including role-gated ones

3.5.1 Money handling

All monetary values are stored as **integer minor units (fils; 1 ETB = 100 fils)**. Floating point is never used. The fare engine rounds the per-kilometre component to whole fils so no float is retained in a monetary total. Conversion to ETB happens only at the presentation edge.

3.5.2 Time, identity and freshness

Concern	Convention	Why it matters
Time	UTC everywhere; rendered <code>Africa/Addis_Ababa</code> at the edge	Ethiopia has no daylight saving; a single clock avoids DST-era fraud windows
IDs	Prefixed, sortable, non-enumerable (<code>TKT-</code> , <code>PAY-</code> , <code>VAL-</code> , <code>CRD-</code>)	A timestamp prefix sorts by creation without reading the timestamp column
Freshness	Every live reading carries <code>REAL_TIME</code> / <code>ESTIMATED</code> / <code>SCHEDULED</code> / <code>STALE</code> / <code>UNKNOWN</code>	A stale position must never be presented as a live one

3.6 Realised reference network

The seeded dataset is real Addis Ababa geography, not placeholder text:

Attribute	Value
Stops	15, with Amharic names and WGS-84 coordinates
Zones	Z1 Piazza core, Z2 Merkato/Derg, Z3 Bole/Kality, Z4 Kirkos/Gefersa
Routes	6 (R-3, R-4, R-5, R-7, R-12, R-14), 4–7 stops each, cumulative distance in metres
Operators	2 licensed operators with licence numbers and settlement accounts
Modes	Bus (live), Taxi and Train (data models present)
Fare rules	Zone-pair matrix (Z1↔Z2, Z1↔Z3, Z2↔Z3), long-distance band, citywide fallback
Concessions	Student 50%, Senior 40%, Disability 100%, all requiring proof
Transfers	80% retained on second leg within 60 minutes, 1 transfer maximum

Stops include Piazza (ፓዥዛ), Derg Square (ደርግ አደካ), Merkato (መርከቶ), Bole (ቦሌ), Kirkos (ክርቆስ) and Kality (ካሊቲ), with shelter and zone attributes. The pilot recommendation in §1.6 targets this corridor because the data exists and demand density is highest.

3.7 Cryptographic credential design

3.7.1 Why Ed25519

Passenger ticket QR codes must be verifiable **offline** by staff handhelds with intermittent connectivity. That operational constraint — not preference — drove the algorithm choice. Recorded as ADR 0002.

Property	Value	Consequence
Public key	32 bytes	Fits anywhere; distribution to 1,000s of handhelds is practical
Signature	64 bytes	QR code remains scannable; an RSA-2048 signature (~294 bytes) would bloat every code
Verification speed	A handful of field operations	Fast enough on a low-end handheld with no crypto accelerator
Determinism	Same payload always yields the same signature	Removes an entire class of replay-with-mutation questions

RSA was the rejected alternative: signature size scales with key size, verification cost scales with key size, and QR capacity is scarce.

3.7.2 What the QR contains — and what it deliberately does not

```
{
  "t": <ticketId>,
  "c": <credentialId>,
  "i": <issuedAt>,
  "e": <expiresAt>,
  "n": <nonce>,
  "k": <keyId>,
  "s": <signature>
}
```

The payload carries **identifiers, timestamps, a 128-bit nonce and a signature**. It carries **no fare, no user id, no route, no name** (invariant I4).

Why this matters practically: a photograph of a passenger's QR reveals nothing about them. It cannot be replayed for a different fare, because the fare is not in it. A screenshot shared socially is a photo of a random reference number.

The credential is a *reference*, resolved by the validator against the stored ticket. The public key is embedded in staff devices at enrolment, so offline validation is cryptographically real rather than a local boolean check.

3.7.3 Canonical serialisation

The signed bytes are produced by joining a **fixed, versioned field order** with a delimiter — never by `JSON.stringify` of an object literal. JavaScript object key ordering is not contractually stable across engines, so a signing/verification mismatch would reject valid tickets. The credential carries an explicit version field (`v`), and an unrecognised version fails closed as `UNSUPPORTED_VERSION` rather than being guessed at.

3.7.4 Key lifecycle

- Private signing key is held by the server only; it is **never** distributed to staff devices.
- Public verification keys are provisioned at device enrolment and can be revoked (`OfflineKey.revokedAt`), supporting lost-device response.
- Every credential records **which key signed it** (`keyId`), so a verifier may hold several public keys and continue to accept historical tickets signed by a retired key. Rotation is therefore non-breaking — a requirement the Bureau should insist on, because a forced bulk reissue after a rotation would be operationally unacceptable.

3.8 How invariants I1 and I2 are enforced in code

Invariants are not documentation. They are structural properties of the code.

3.8.1 I1 — a ticket exists only against a confirmed payment

Enforced from three independent directions, so removing any one still leaves a barrier:

1. **The orchestrator cannot issue tickets.** The payment orchestrator returns payment state only. There is no method that mints a ticket, so "payment succeeded" and "ticket issued" are not conflated by construction.
2. **The state machine has exactly one edge into** `TICKET_ISSUED`, from `PAYMENT_CONFIRMED`. A test iterates every status and asserts that none other can reach it.
3. `assertIssuable(ticketStatus, paymentStatus)` refuses anything that is not `CONFIRMED`, raising an error that names the I1 violation explicitly.

The specific failure prevented: treating a client returning from a payment redirect as proof of payment. `REQUIRES_ACTION` carries a redirect URL and is explicitly *not* confirmation; there is a dedicated test for exactly that.

3.8.2 I2 — idempotency is a database constraint

The guarantee is a `UNIQUE` index on `payments.idempotencyKey`. Everything else is an optimisation layered on top:

- `createPayment` checks for an existing key first (fast path).
- If that check misses but a concurrent request wins the race, the insert raises a unique violation, which is caught and the **winning** payment is returned.
- `initiate` never re-opens a payment in a terminal state, so a replayed request cannot restart a settled or failed payment.

Parallel double-taps therefore converge on **one charge** rather than surfacing an error to a passenger who tapped twice — the correct behaviour at the point of sale.

3.9 Authorisation model

3.9.1 Twelve staff roles

`DRIVER`, `CONDUCTOR`, `INSPECTOR`, `TICKET_OFFICER`, `AGENT`, `SUPERVISOR`, `OPERATOR_ADMIN`, `TRANSPORT_BUREAU_ADMIN`, `FINANCE`, `AUDITOR`, plus `SUPER_ADMIN`. Every role list the API authorises against is declared in **one file**, beside the guard that enforces it.

3.9.2 The operator boundary — a role grant is not sufficient

Staff may act only on their own operator's resources unless they hold a bureau or finance role. This boundary is enforced **in the service layer** by an explicit assertion, not merely by which buttons the UI shows. An `OPERATOR_ADMIN` editing their own fleet is filtered by data ownership as well as by role.

This is the control that makes multi-operator rollout safe. Without it, the first operator with a compromised credential can write to every other operator's network.

3.9.3 Role-scoped permissions

Capability	Roles permitted
View dashboard	Bureau admin, finance, auditor, operator admin, supervisor
View money (revenue, fares)	Bureau admin, finance, auditor, super admin

Capability	Roles permitted
Manage network data	Bureau admin, operator admin, super admin
Draft a fare change	Bureau admin, finance, super admin
Approve a fare change	Bureau admin, finance, auditor, super admin
Change a staff role	Bureau admin, super admin only
Scan / validate tickets	Inspector, ticket officer, supervisor, agent, conductor, operator admin, bureau admin
Read database internals	Bureau admin, super admin only

Two design decisions deserve the Bureau's attention:

- **Field roles are excluded from the dashboard entirely** rather than shown a reduced view. A half-visible figure is still a leak, and a dashboard they can never use is not a capability anyone requested.
- **Fare author and fare approver role lists deliberately overlap** — the same agency approves its own fares. Overlap alone does not grant self-approval, because the service layer separately **forbids a draft's own author from activating it**. This is dual control implemented so that role configuration cannot accidentally defeat it.

3.9.4 Reporting, not enforcing, on the client

The session endpoint reports what a given role may do, so the app can hide panels that would only return 403. This is **presentation, not security**: the server re-reads the role from the database and enforces every rule on every request.

3.10 Data model summary

~40 models. The five invariants are expressed as schema constraints, so they hold regardless of application behaviour.

Domain	Principal models
Identity	User, Passenger, Staff, Operator, Device, StaffDevice, OtpChallenge, RefreshToken
Mobility	Route, Stop, RouteStop, Vehicle, Trip, Driver, Station, StationGate, TrainRoute
Fare	FareRule (versioned), FareCalculation, Concession, TransferRule
Journey	Journey, JourneyLeg
Payment	Payment, Wallet, Settlement
Ticketing	Ticket, QrCredential, TicketValidation, VehicleQrCredential
Field ops	Shift, Trip, Manifest
Assurance	Complaint
Governance	AuditEvent (append-only, hash-chained), Notification

3.10.1 Security-relevant schema decisions

Decision	Reason
OTP codes stored only as a hash with a per-challenge salt	A database leak must not yield usable login codes. Six digits is ~1M possibilities, trivially brute-forceable offline without a salt
Refresh tokens stored as hashes only , revocable per device	One revocation per lost device rather than a global credential reset

Decision	Reason
Money as integer fils with <code>Decimal</code> coordinates	Never floating point for money; <code>Decimal(9,6)</code> for coordinates
Foreign keys with cascade only where deletion is intended	Ticket history must not vanish with a user record
<code>idempotencyKey</code> UNIQUE	Invariant I2 at the database layer
<code>fareRuleVersion</code> NOT NULL on Ticket	Invariant I3 at the schema layer
<code>QrCredential</code> holds identifiers only	Invariant I4 at the schema layer
<code>AuditEvent</code> has no update path in application code	Invariant I5; corrections are new rows

Part IV — Technical Procedures

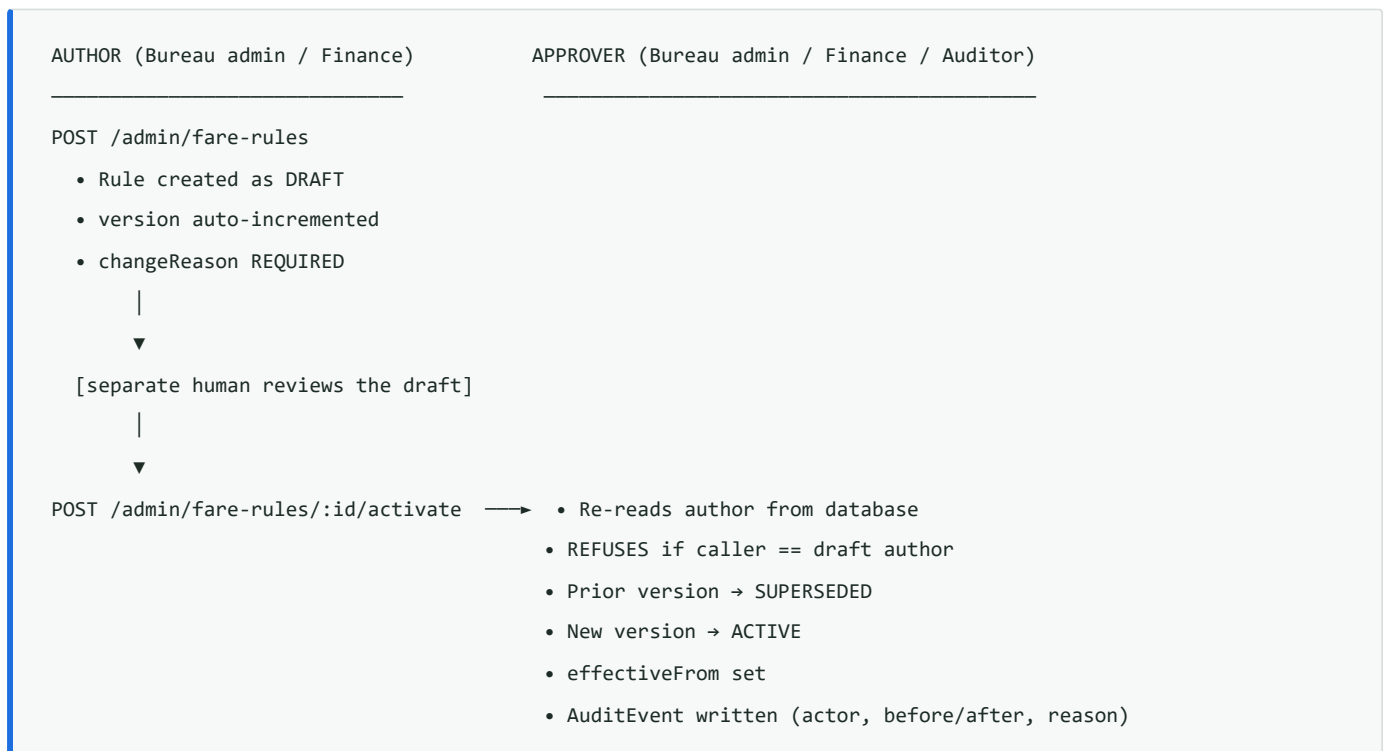
These are the operating procedures the Bureau will run day to day. Each states the **trigger**, the **actor**, the **system behaviour**, the **control evidence produced**, and the **failure handling**.

Every endpoint below is live in the inspected build. Where a procedure requires a component that is specified but not yet implemented, it is marked **[Phase 2]** and described honestly.

4.1 Procedure 1 — Fare rule drafting, approval and activation

Purpose: ensure every price a passenger is charged is traceable to an approved, versioned policy, and that no single person can change it alone.

4.1.1 Sequence



4.1.2 Control rules

Rule	Enforcement
A fare rule cannot be edited in place	Revising creates a new version ; the old version is retained
The draft's author cannot approve it	Checked in the service layer against the persisted author id
Approval requires a stated reason	<code>changeReason</code> is required and stored
Every version change is audited	<code>AuditEvent</code> with old and new values

4.1.3 Why versioning is non-negotiable

Without it, a fare revision silently rewrites history: a passenger disputing a charge from eight months ago cannot be shown what the fare *was* at the time, because the number has changed. With versioning, every ticket pins the exact rule version used, so any historical fare is reproducible from stored inputs — for years, and across administrative turnover.

4.1.4 Failure handling

A fare change that would leave **no active rule** for a served zone-pair is rejected. The Bureau cannot accidentally fare a route to zero or to infinity; a gap is visible as a coverage report before it becomes a passenger dispute.

4.2 Procedure 2 — Fare calculation

Purpose: compute the correct fare from published policy, server-side, and produce a record that explains itself.

4.2.1 Selection algorithm

Fare rules are ranked by **specificity**, most specific first:

Factor	Score added
Route-specific rule	+8
Operator-specific rule	+4
Origin and destination zone	+4
One zone only	+2
Distance band	+2

Final score is `specificity × 1000 + version`, so at equal specificity the **later version** wins. A route-and-zone rule therefore beats a citywide zone rule, which beats a bare citywide mode rule.

4.2.2 Calculation sequence

1. Filter rules: must be ACTIVE, effective at the travel time, and matching mode / route / operator / zones / distance band.
2. Rank by specificity; take the highest.
3. `distanceFare = round(perKmFils × distanceKm)` ← rounded to whole fils
4. `subtotal = baseFare + distanceFare`
5. Apply concession discount (if an approved concession is held)
6. Apply transfer discount (second leg, if within the transfer window)
7. `floored = max(subtotal, minimumFare)` ← floor BEFORE discount
8. `total = max(0, floored - discount)`

Ordering rule that matters: the minimum-fare floor is applied **before** discounts. Applying it afterwards would allow a 100% concession on a below-minimum fare to drive the result negative. Non-negativity is then clamped as a final belt-and-braces guard.

4.2.3 The returned breakdown

Every calculation returns an explanation, not just a number:

Rule ZONE-1-3 v2 (BUS) applied:

base 1500 + distance 786 - discount 0 = 2286 fils

This is what allows a Bureau officer answering a fare dispute to show *why* a passenger was charged a specific amount.

4.2.4 Failure handling — fail closed, visibly

If no rule matches, the engine does not invent a price. It applies a configurable fallback base, sets `usedFallback: true`, and records an explanation naming the condition. This is deliberate: inventing a fare would be worse than surfacing a zero, and the flag guarantees the event is visible to monitoring rather than being a silent wrong charge. The Bureau should treat a rising fallback rate as a fare-coverage alarm.

4.3 Procedure 3 — Journey planning

Purpose: give a passenger a correct, affordable, explainable route, and persist the quote so the purchase can be verified against it.

4.3.1 Sequence

```

TRIGGER Passenger selects origin + destination (or accepts a GPS "nearest stop")
|
|→ GET /stops/nearby?lat&lng&radius
|
|    Server ranks ALL stops by great-circle distance.
|
|    (Deliberately server-side: a device-side search would rank only the
|    25 stops that happened to sort near "A", snapping a passenger to
|    the start of the alphabet instead of the stop they are standing at.)
|
|→ POST /journeys/plan { originStopId, destinationStopId, departureTime }
|
|    Build candidates:
|
|        • DIRECT: any route serving both stops in the correct direction
|        • TRANSFER: one interchange via a shared stop
|
|    Price each leg through the SHARED fare engine
|    (never recomputed – one implementation of quoting rules)
|
|    Apply transfer rule (e.g. 80% retained on the 2nd leg within 60 min)
|
|    Rank by total fare, then duration
|
|    PERSIST the best-ranked option as Journey + FareCalculation
|
|        Returns journeyId – this is what the client must
|        send to buy, because purchase re-verifies the amount
|        against the stored calculation

```

4.3.2 Why only the top option is purchasable

The response returns multiple options, but **only the persisted, best-ranked option carries a `journeyId`**. The others were quotes the passenger declined; they have no stored fare calculation, so there is nothing to verify a payment against. A quote that was never written down cannot be checked — so the client must re-plan to buy.

This is a deliberate constraint, not an inconvenience: it removes any possibility of a client naming a price the server never computed.

4.3.3 Deliberate simplicity

The planner is direct-ride-plus-one-transfer, not a shortest-path search over a full timetable. The stated reason in the code is worth repeating: **when a passenger disputes a fare, the option must be explainable**. An opaque algorithm cannot be defended to a citizen. A simple one can be shown on screen.

`freshness` is returned on every journey and is currently `ESTIMATED`, never `REAL_TIME`, because no live GPS feed is wired up. **[Phase 2]** connects the AACRA ITS feed; the field already exists so that downstream consumers must handle freshness explicitly rather than assuming real-time.

4.3.4 Timings

Durations derive from route distance at a constant average speed plus a fixed boarding allowance per stop. These are placeholders calibrated in the pilot (§7.3). The Bureau should expect to replace them with ITS-sourced actual running times.

4.4 Procedure 4 — Purchase, payment and ticket issuance

Purpose: take a passenger's money exactly once and issue exactly one valid ticket, through one path regardless of payment method.

4.4.1 The critical control — the fare is re-derived, not trusted

```
Client POST /payments {  
  journeyId, amountFils, method, idempotencyKey, payerPhone?  
}
```

|



```
| SERVER: re-derive the fare from the stored FareCalculation |  
| The client-sent amount is treated as a CLAIM to be        |  
| CHECKED, never as the price.                             |  
| A passenger could otherwise POST amountFils:1 for a      |  
| 44-birr ride – and the provider would charge one cent.   |
```

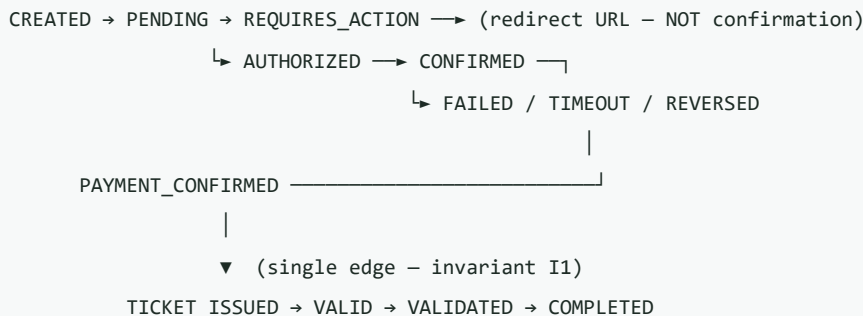
|



```
Amount mismatch → REJECTED (no charge created)
```

This is the single most important line of defence against fare theft at the API, and it is why a modified client cannot buy a 22.86 Birr ticket for 1 cent.

4.4.2 Payment state machine



4.4.3 Idempotency — the double-tap contract

The client generates an `idempotencyKey` **once**, before the first attempt, and reuses it for every retry of the *same* purchase. A new purchase gets a new key.

Situation	System behaviour
Passenger double-taps Pay	Both requests carry the same key → one charge , the second returns the first payment
Network drops the response, client retries	Same key → original payment returned, no second charge
Two devices race on the same key	Unique-violation caught, winning payment returned to both
Client replays a settled payment	<code>initiate</code> refuses to re-open a terminal state
Two parallel requests on a fresh key	In-memory test repository deliberately reproduces the unique violation, so tests exercise the same race the database enforces

4.4.4 Provider adapter contract

Rule	Reason
Stale webhooks cannot undo a settled payment	Terminal states are immutable on webhook application
Non-authoritative states (<code>PENDING</code> , <code>AUTHORIZED</code>) are ignored	Only a terminal webhook advances the payment
Over-capture and over-refund are rejected , not clamped	Clamping would mask an integrator defect
Unknown provider raises , never silently no-ops	A missing adapter must be a loud failure, not a free ticket

The last row deserves emphasis for a revenue system: **silently succeeding when the payment integration is missing is the same failure mode as handing out free tickets**. The system is built to fail loudly instead.

4.4.5 Ticket issuance

On `CONFIRMED`, the system issues the ticket and signs a credential. Three properties make the credential safe:

- **Deterministic signature** — the same payload always yields the same signature.
- **Stored nonce** — the ticket history re-derives the QR from stored columns and **never re-signs**. A fresh signature would carry a new nonce, invalidating any screenshot the passenger already holds and looking like a replay to a validator.
- **Explicit version field** — an unknown credential version is rejected rather than interpreted.

4.4.6 Assisted (cash and agent) sales

Cash sales flow through the **same orchestrator** as mobile payments, producing the same core records, with `collectedByStaffId` and `shiftId` attached. This is what makes shift reconciliation possible — an agent sale that bypassed the core would be revenue with no audit trail, and trivially forgeable.

Two controls on the assisted path:

- The officer **cannot nominate a price**. Fare re-derivation is identical.
- `shiftId` and `collectedByStaffId` are taken from the **authenticated officer and their open shift**, never from the request body. Anything else would let one officer file revenue against another's drawer.

A cash sale is refused outright when no shift is open.

4.5 Procedure 5 — Ticket validation (boarding inspection)

Purpose: determine, at the door, whether a passenger may board — and record the answer whether it is yes or no.

4.5.1 Sequence

```

INSPECTOR  scans passenger QR with handheld
|
▼
POST /validation/scan { qrString, tripId?, deviceId?, lat?, lng? }
|
├ 1. Parse credential → MALFORMED / UNSUPPORTED_VERSION → reject
├ 2. Select public key by keyId → UNKNOWN_KEY → reject
├ 3. Verify Ed25519 signature → BAD_SIGNATURE → reject
├ 4. Check time bounds → EXPIRED / NOT_YET_VALID → reject
├ 5. Load stored ticket → NOT_ISSUED / UNKNOWN_CREDENTIAL → reject
├ 6. Check ticket status (state machine) → ALREADY_USED / REVOKED → reject
├ 7. Check trip and mode binding → WRONG_TRIP / WRONG_MODE → reject
└ 8. Accept → ticket transitions VALID → VALIDATED
    |
    ▼
    EVERY outcome written to TicketValidation, including refusals
  
```

4.5.2 The outcome vocabulary

Eleven distinct outcomes are defined, so that an officer is never given a confusing code. This matters because **an officer told `NOT_ISSUED` when the truth is `EXPIRED` sends a paying passenger away from a vehicle they were entitled to board**. The mapping is a deliberate lookup table rather than nested conditionals, because this is the one place where a confusing code does real damage.

Cryptographic outcome	Reported as	Officer-facing meaning
<code>MALFORMED</code> , <code>UNSUPPORTED_VERSION</code> , <code>NOT_YET_VALID</code>	<code>INVALID</code>	Not a readable ticket
<code>BAD_SIGNATURE</code>	<code>INVALID_SIGNATURE</code>	Forged or altered — escalate
<code>EXPIRED</code>	<code>EXPIRED</code>	Outside its validity window

Cryptographic outcome	Reported as	Officer-facing meaning
UNKNOWN_KEY	UNKNOWN_CREDENTIAL	Not issued by this system

4.5.3 Refusals are recorded, not discarded

Every scan is written to the database, including every rejection. A fraud pattern is only visible if refusals are recorded with the same fidelity as acceptances. A failed scan is therefore not an error path that quietly returns 4xx and leaves no trace — it is a row with outcome, reason, officer, device, trip and coordinates.

4.5.4 The HTTP status decision

Validation returns **HTTP 200 even for a rejection**. The HTTP status answers "did the server accept your request?"; the `accepted` field answers "may this passenger board?" Collapsing the two would force every handheld to treat an invalid ticket as a network error, and would fill the server log with 400s that look like client faults when they are in fact successful fraud detections.

4.5.5 Availability principle — the officer's answer comes first

If the audit write fails, the system **logs loudly and still returns the refusal**. The officer standing at the door needs an answer more than the system needs an audit row. The reference is returned as `UNRECORDED` so the gap is visible rather than silent.

4.5.6 Offline operation

[Phase 2 — schema and client queue ready, server reconciliation pending]

Design commitment, which the Bureau should require in any acceptance test:

1. The handheld holds the **public key only** and verifies the Ed25519 signature **locally**. Offline validation is cryptographically real, not a local boolean.
2. Results queue locally and sync when connectivity returns.
3. The server reconciles queued scans, resolving conflicts by the ticket state machine — a second scan of an already-validated ticket is `ALREADY_USED`, never a double validation.
4. Each queued record carries `isOffline` and `syncState` so the Bureau can distinguish online from reconciled-offline validations in reporting.

A clock-drift endpoint already exists so that handheld clock skew can be measured rather than guessed at.

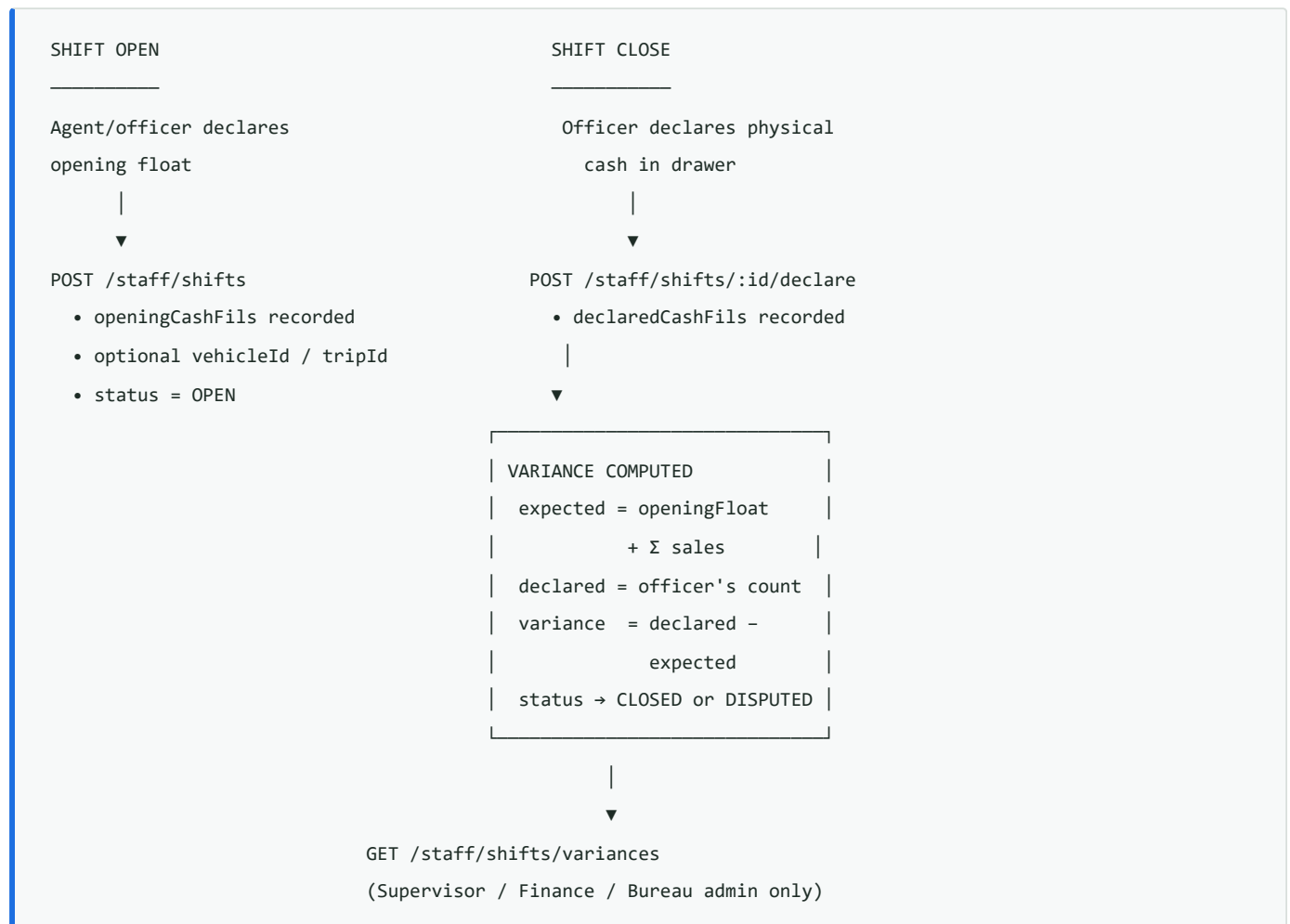
4.5.7 Privacy limit on inspection history

An officer's scan history returns **only that officer's own scans**. A network-wide feed of who inspected whom, where and when is surveillance data with no operational use on a handheld. Oversight queries belong to the audit surface, under Bureau control. The Bureau should note this design decision, because it constrains how the system can be used for staff performance monitoring.

4.6 Procedure 6 — Shift and cash reconciliation

Purpose: bring cash revenue under a control environment. This is the procedure that closes the gap identified in §2.1.2, and it is the Bureau's strongest financial-control argument for the programme.

4.6.1 Sequence



4.6.2 Control properties

Property	Implementation	Why it matters
Opening float must be ≥ 0	Validated at the boundary	A negative float would invert the variance calculation, turning a missing float into an apparent surplus
One open shift per officer	Checked before opening	Otherwise a sale could be attributed ambiguously
Sales refused with no open shift	<code>requireOpenShift</code>	No revenue without a drawer to reconcile against
Officer cannot nominate the shift	Taken from the authenticated principal	One officer cannot file revenue against another's drawer
Variance visibility is role-narrowed	Supervisors and finance only	Cash shortfalls are a supervisory matter, not every officer's business
Declaring is a financial fact , not a view	Returns HTTP 201	A variance may now exist that did not before

4.6.3 The idempotent-replay trap — and why it is handled

Consider: an officer's tablet retries a cash sale while their shift has already been closed and a new one opened. If the system echoed the *currently open* shift, the officer would be told their new drawer is holding a sale recorded against an earlier one. That sale then fails to reconcile and **looks like missing money** — creating a false fraud signal from a correct system.

The system therefore returns the shift the **payment actually belongs to**, and reports the shift reference only when it matches the current one. This is a small detail with a disproportionate effect on operator trust.

4.6.4 What the Bureau gains

Before: cash shortfalls were discovered by suspicion or expensive after-the-fact audit.

After: **every shift has an opening float, a sales total, a declared figure and a variance**, all attributable to a named officer and time window. The variance trend becomes a measurable operational indicator, and it becomes possible to distinguish systematic process weakness from individual dishonesty — because there is finally a baseline to compare against.

4.7 Procedure 7 — Complaints and service assurance

Purpose: convert anecdotal service-quality reporting into structured, geolocated, measurable data.

4.7.1 Sequence



4.7.2 The eight complaint categories

`SERVICE_QUALITY`, `OVERCHARGE`, `REFUND_REQUEST`, `SAFETY`, `DISCRIMINATION`, `LOST_PROPERTY`, `CLEANLINESS`, `OTHER`.

`OVERCHARGE` is the category that connects assurance directly to revenue control: a confirmed overcharge is simultaneously a refund obligation and evidence of fare non-compliance on a specific route or operator.

4.7.3 Design decisions worth noting

- **Citizens can complain without being staff.** The filing route is deliberately the one route without a staff guard. A citizen must be able to complain without holding a staff account. It is still marked public, so an anonymous caller gets a **reference** back rather than a bare 401.
- **Signed-in complaints attach to the passenger** and appear in their history, because a token is read when present even on a public route.
- **Description length is bounded at both ends.** A floor stops "no" becoming a record that wastes an officer's time; a ceiling stops a pasted document filling a column.
- **Location capture is optional**, because requiring GPS would suppress complaints from passengers without data service or battery.

4.8 Procedure 8 — Identity, authentication and session security

4.8.1 Citizen sign-in

1. POST /auth/request-otp { phone } phone validated as canonical Ethiopian mobile format (+251[79]XXXXXXX)
2. Six-digit code delivered by SMS
3. POST /auth/verify-otp { phone, code }
 - codeHash + per-challenge salt compared
 - attempt counter, max 5, then challenge invalidated
 - expiry enforced
 - account found or provisioned (User + Passenger together)
 - JWT + refresh token issued

There is no separate staff login. Staff sign in through the identical OTP flow and hold an identical JWT; what separates a citizen from staff is `StaffGuard`, which checks for an active, unterminated `Staff` record on every request. One credential store, one token path — one thing to revoke, one place a bug can hide.

4.8.2 Security properties

Property	Implementation
OTP codes stored as hash + per-challenge salt	A database leak yields no usable codes; the salt defeats precomputation over the ~1M six-digit space
Brute force limited	Attempt counter with a maximum, then the challenge is destroyed
Refresh tokens stored as hashes , revocable per device	A lost device is revoked individually, not by a global credential reset
Account enumeration resisted	Identical refusal text whether a number is unknown or simply not staff
MFA supported for privileged staff	<code>mfaEnabled</code> / <code>mfaSecretEnc</code> on <code>Staff</code> for high-risk permissions
Staff role re-read from database per request	A demotion takes effect immediately, not at token expiry

4.8.3 Development backdoor — deliberately hard to leave open

A development-only sign-in exists that skips the SMS round trip. It is gated by **both** `DEV_STAFF_LOGIN=true` and `NODE_ENV ≠ production`, because either condition alone would be dangerous: the flag alone would open it on a deployed environment that merely inherited the `.env`; the environment check alone would open it on a production build pointed at a development database.

It also **never creates a `Staff` row** — it requires an existing, active, unterminated one. It is a shortcut past the SMS round trip, not a way to mint an administrator. When disabled, the route is **never registered**, so it answers 404 rather than existing-but-hidden; a capability hidden in the UI while a live endpoint remains behind it would be worse than having neither. Every use is logged loudly.

Recommendation to the Bureau: require this route to be absent in the pilot deployment, and verify by requesting it and confirming a 404.

4.9 Procedure 9 — Governance reporting

Purpose: give Bureau officers a live, role-scoped view of the network.

4.9.1 Reporting surfaces

Endpoint	Content	Restricted to
/dashboard/session	Who am I, what may I see	Any dashboard role
/dashboard/overview	Headline operational and revenue counters	Dashboard roles
/dashboard/revenue	Paginated revenue detail, filterable by status	Finance roles only
/dashboard/operations	Service volumes, trips, validations	Dashboard roles
/dashboard/network	Route, stop, vehicle, operator inventory	Dashboard roles
/dashboard/fares	Fare rule versions and current policy	Finance roles only
/dashboard/passengers	Passenger volume trends	Dashboard roles
/dashboard/database	DB size, row counts, pool, index usage	Bureau admin only

4.9.2 Why the restrictions are shaped this way

- **Operational counts and money are separated.** An operator admin legitimately needs to see their own takings, but a supervisor needs service volumes, not other people's margins.
- **Database internals are narrower than everything else.** Table names, index names and connection counts describe how the system is *built* rather than how it is *performing*, and are useful mainly to someone about to change it. An auditor can see the operational figures they are attesting to; only administration roles see the plumbing behind them.
- **The dashboard is read-only by construction** — no write endpoints exist on the controller at all.

4.9.3 Operator scoping

Every query is scoped by the operator boundary. An `OPERATOR_ADMIN` sees their own fleet and revenue; bureau and finance roles see city-wide. This is enforced in the service layer on every query, not by filtering the response after the fact.

4.9.4 Deployment note

The dashboard is served from disk with caching explicitly disabled (`maxAge: 0`), so an officer who has bookmarked the URL is never shown a stale build of the reporting screen. It is same-origin, so the browser sends the bearer token with no CORS preflight and no wildcard origin has to be permitted.

4.10 Procedure 10 — Audit and tamper evidence

Purpose: make privileged actions attributable and make bulk tampering detectable after the fact.

4.10.1 What is audited

Refunds, overrides, voids, fare changes, cash declarations, device lifecycle, role changes, staff sales, complaint resolutions — every action that moves money, changes policy or grants access.

4.10.2 Record structure

Each `AuditEvent` carries: `actorId`, `actorType`, `actorRole`, `action`, `resourceType`, `resourceId`, `timestamp`, `deviceId`, `ipAddress`, `userAgent`, `oldValue`, `newValue`, `reason`, dual-control `approvalId`, and the hash chain.

4.10.3 Two properties that make this useful to an auditor

Append-only. There is no update path in the application for `AuditEvent`. Corrections are new rows. The schema comment is explicit: tamper-evidence via a chained hash, where each event carries `previousHash` and `eventHash`.

Bulk tampering is detectable. Because each event hashes its predecessor, deleting or editing a range breaks every subsequent link. An auditor can therefore verify the *continuity* of the log, not merely the presence of rows. This is a materially stronger guarantee than an ordinary application log.

4.10.4 Dual control on high-risk actions

The schema carries `approvalId` and `approvalByStaffId` explicitly, so an approval-required action is attributable to **both** the requester and the approver. This is what makes the fare-approval workflow auditable as a two-person control rather than a single click.

4.11 API reference summary

All routes are served under `/api/v1`. Authentication is bearer JWT.

Group	Routes
Auth	<code>POST /auth/request-otp</code> , <code>POST /auth/verify-otp</code> , <code>POST /auth/refresh</code> , <code>GET /auth/staff-session</code>
Stops / Journeys	<code>GET /stops</code> , <code>GET /stops/nearby</code> , <code>POST /journeys/plan</code> , <code>GET /journeys/mine</code>
Payments	<code>POST /payments</code> , <code>GET /payments/:id</code> , <code>GET /payments/:id/ticket</code>
Tickets	<code>GET /tickets/mine</code>
Validation	<code>POST /validation/scan</code> , <code>GET /validation/mine</code> , <code>GET /validation/clock</code>
Driver	<code>GET /driver/trips</code> , <code>GET /driver/trips/:id/manifest</code> , <code>POST /driver/trips/:id/start</code> , <code>POST /driver/trips/:id/complete</code>
Staff / Shifts	<code>POST /staff/shifts</code> , <code>GET /staff/shifts/mine</code> , <code>POST /staff/shifts/:id/declare</code> , <code>GET /staff/shifts/variances</code> , <code>POST /staff/sales</code> , <code>GET /staff/devices</code> , <code>POST /staff/devices/:deviceId/enroll</code>
Admin	<code>GET/POST /admin/stops</code> , <code>PATCH /admin/stops/:id</code> , <code>PUT /admin/stops/:id/active</code> , same for <code>vehicles</code> , <code>routes</code> , plus <code>PUT /admin/routes/:id/stops</code> , <code>GET/POST /admin/fare-rules</code> , <code>POST /admin/fare-rules/:id/activate</code> , <code>GET /admin/operators</code> , <code>GET /admin/staff</code> , <code>PATCH /admin/staff/:id</code>
Complaints	<code>POST /complaints</code> , <code>GET /complaints</code> , <code>GET /complaints/mine</code> , <code>PATCH /complaints/:reference</code>
Dashboard	As listed in §4.9.1
Health / Docs	<code>GET /health</code> , <code>GET /api/docs</code> (non-production only)

4.11.1 Input validation policy

A global validation pipe enforces `whitelist: true` and `forbidNonWhitelisted: true` with transformation. Unknown fields are **stripped and rejected**, so a client cannot smuggle extra columns through a DTO into the persistence layer.

Explicitly bounded inputs include: QR strings (20–2048 chars — an inspector's camera produces a few hundred bytes, so anything approaching the bound is not a ticket, and an unbounded string would otherwise be written verbatim into an audit column), phone numbers (canonical Ethiopian format), coordinates (latitude and longitude validators), and all monetary values (integers, minimums applied).

4.12 Honest capability statement

A proposal that overstates what exists loses the Bureau's trust at the first technical review. This section states precisely what is built, what is partial, and what is not started.

4.12.1 Fully implemented and tested

Capability	Evidence
Fare engine with specificity ranking, versioning, concessions, transfers, floors	<code>fare-engine.ts</code> + 21 tests
Ticket state machine with single-edge issuance	<code>ticket-state-machine.ts</code> + 24 tests
Payment orchestrator with idempotency, terminal-state immutability, provider contract	<code>payment-orchestrator.ts</code> + 23 tests
Ed25519 credential signing and verification with canonical serialisation	<code>credential-signer.ts</code> + 18 tests
Centralised 12-role authorisation policy with operator boundary	<code>policy.ts</code> + 25 tests
Journey planning with direct and one-transfer routing	<code>journey-planner.service.ts</code>
Validation scan pipeline with 11-outcome vocabulary and refusal recording	<code>validation.service.ts</code>
Shift and cash reconciliation with variance	<code>identity.controller/service.ts</code>
Complaints workflow with 8 categories	<code>assurance.controller/service.ts</code>
Governance dashboard, read-only, role-scoped	<code>dashboard.*</code> + 20 tests
OTP auth, hashed codes with salt, revocable refresh tokens	<code>auth.*</code>
Admin CRUD for stops, vehicles, routes, fare rules, staff	<code>admin.*</code>
~40-model schema with invariants as constraints	<code>schema.prisma</code>
Docker Compose for PostgreSQL, Redis, TimescaleDB	<code>infra/docker-compose.yml</code>

4.12.2 Implemented but using a mock provider

The payment integration currently runs against a **mock adapter**, not live Telebirr or CBE Birr. The orchestration, state machine, idempotency and issuance guarantees are fully real and tested — but **no live money has moved through the inspected build**.

Live provider integration is **Phase 1 pilot work** (§6.2), and requires commercial agreements with the providers themselves, which is outside the software scope.

4.12.3 Partially implemented

Capability	State	Needed
Offline validation	Staff app has a local scan queue and offline flag fields; the server-side reconciliation endpoint does not exist	Reconciliation endpoint + conflict resolution by state machine
Live vehicle tracking	Time-series store provisioned and <code>DataFreshness</code> enum defined; no ITS ingestion	AACRA ITS feed integration
Real-time journey freshness	Always returns <code>ESTIMATED</code>	Timetable or ITS-derived timings
Refunds	Payment states and ticket states exist; no refund endpoint exposed	Refund workflow with dual control

Capability	State	Needed
Settlement to operators	<code>Operator.settlementAccount</code> and <code>Settlement</code> model exist; no settlement engine	Settlement runs
Wallet top-up	<code>Wallet</code> model exists	Top-up flows
Concession proof verification	Concessions modelled and applied	Document/ID verification workflow
Notification delivery	<code>Notification</code> model exists	Delivery integration

4.12.4 Not started

- **Taxi and rail modes** — data models present, journey planner handles bus only
- **Native Amharic UI** — Amharic stop and route *names* are stored and served; interface strings are English
- **Multi-language voice/announcement** for accessibility
- **Machine-readable open-data portal** for the public
- **Offline-first ticket wallet with pre-fetched credentials**

4.12.5 What we ask the Bureau to require as acceptance evidence

Rather than take our word for any of this, the Bureau should require:

1. **Automated test results** for all 141 tests (7 suites, all passing), run in the Bureau's presence.
 2. **A live demonstration** of the fare dispute workflow: issue a ticket, revise the fare, and show that the old ticket still quotes the old rule version.
 3. **A tamper-evidence demonstration**: alter or delete an audit row, and show that chain verification fails.
 4. **A double-tap demonstration**: fire two concurrent purchase requests and observe one charge.
 5. **A modified-client demonstration**: attempt to purchase a ticket with a falsified amount and observe rejection.
 6. **Independent penetration testing** before any citywide rollout.
-

Part V — Governance, Security and Data Protection

5.1 Security architecture

5.1.1 Defence in depth

Layer	Control
Edge	TLS termination, Helmet security headers, explicit CORS allow-list, rate limiting via Redis
Authentication	JWT bearer tokens, OTP for citizens, MFA supported for privileged staff, hashed refresh tokens
Authorisation	Two-guard chain — <code>JwtAuthGuard</code> then <code>StaffGuard</code> — then role, then operator boundary in the service layer
Input	Global <code>ValidationPipe</code> with whitelist and forbid-non-whitelisted; explicit length and range bounds on every field
Cryptography	Ed25519 signing, salted hashing for OTP codes, private keys server-side only
Data	Integer money, constrained schema, foreign keys, cascade only where intended
Audit	Append-only, hash-chained, actor/device/IP/reason on every privileged action

5.1.2 The two-guard pattern

Staff routes apply `JwtAuthGuard` **then** `StaffGuard`, in that order, and only then check roles. The first establishes *who is calling*; the second establishes *that they are staff and which role they hold*.

This matters because a citizen's token is a valid API token. Without the second guard, a citizen could reach revenue data with their own valid token. The Bureau should verify in testing that a citizen token receives 403 on every `/dashboard/*` route.

5.1.3 Configuration safety

Environment configuration is validated **before** anything touches the database, with a guard that names the offending variable, names any competing source, and **redacts the password**. This exists because Prisma's own error for a bad connection string points an operator at `schema.prisma` rather than at the environment variable they actually set — a costly misdiagnosis in production.

`.env` is loaded explicitly at startup, before the guard runs, because the Nest config module only reads it while building the dependency-injection container — several lines later. Without the explicit load, the guard would reject a perfectly good configuration file. Operator-exported variables are deliberately **not** overridden by the file, because a silently preferred `.env` would hide exactly the mislabelling this guard exists to diagnose.

5.2 Data protection and privacy

Ethiopia's Personal Data Protection Proclamation and the Bureau's own data governance expectations should govern this programme. The system's design already supports the major requirements:

Principle	Implementation
Data minimisation	QR payloads contain no personal or commercial data (I4); a photograph of a ticket reveals nothing about its owner
Purpose limitation	An officer can retrieve only their own scan history; network-wide inspection surveillance is deliberately not exposed to field roles
Storage limitation	Refresh tokens hashed and revocable; OTP codes hashed with per-challenge salt and short expiry

Principle	Implementation
Access control	12 roles, operator boundary enforced in the service layer, city-wide read restricted to bureau and finance
Integrity	Hash-chained audit log makes tampering detectable after the fact
Accountability	Every privileged action attributable to actor, device, IP, time and stated reason

5.2.1 Points requiring legal review before citywide rollout

We flag these rather than assert compliance:

1. **Retention periods** for journey, ticket, validation and audit records — the system stores them indefinitely today.
2. **Passenger location data** — GPS is captured for nearest-stop search and optionally on complaints. Retention and aggregation rules need a decision.
3. **Concession and national ID storage** — `Passenger.nationalId` and `concessionId` are modelled. Whether to populate them at all is a policy decision, and the system works without them.
4. **Cross-border data residency** — deployment location determines jurisdiction.
5. **Disclosure to operators** — how much of a passenger's journey history an operator may lawfully see.

5.2.2 Recommended privacy commitments

We propose the Bureau adopt these as programme policy:

- **No commercial data in any scannable credential**, permanently.
- **No network-wide movement surveillance** of individual passengers; aggregate OD data only, with a minimum cell size.
- **Officer scan history is the officer's own**, with oversight queries confined to the audit surface under Bureau control.
- **Purge or anonymise journey data** on a published schedule after the reconciliation window closes.

5.3 Sovereignty and lock-in

The Bureau's institutional capability must outlive any vendor.

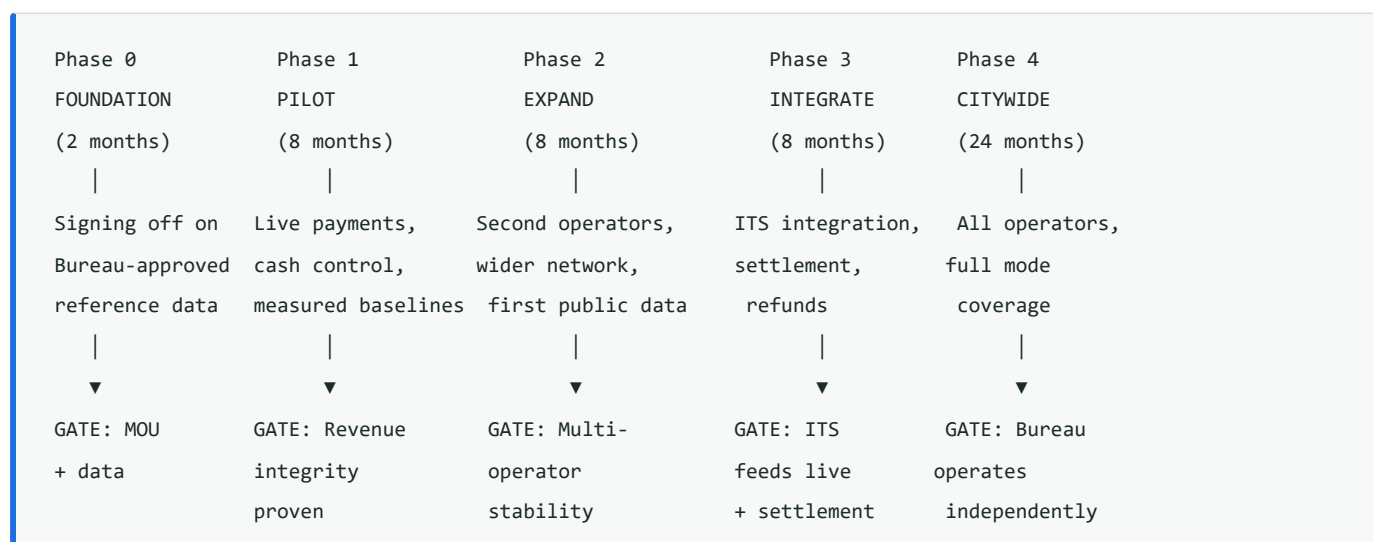
Asset	Bureau position
Source code	Fully in the Bureau's environment, no proprietary runtime dependency
Database	PostgreSQL — Bureau-owned instance, standard backups, PITR
Data export	Full SQL export; no encrypted silo the Bureau cannot read
Keys	Bureau-held signing key custody; documented rotation procedure
Standards	REST/JSON, OpenAPI 3, Ed25519 — all documented, all standard
Skills	Bureau staff trained as administrators (\$7.5)
Exit	Documented handover and escrow; the platform remains operable if the Team departs

We deliberately reject proprietary wire formats and closed payment protocols. A public revenue system the Bureau cannot independently operate is not a long-term asset.

Part VI — Implementation Roadmap

6.1 Phasing philosophy

The sequence is driven by **evidence, not calendar**. Each phase ends with a decision point where the Bureau can stop, extend or proceed on measured grounds. We specifically avoid a plan that assumes citywide success and works backwards to justify it.



6.2 Phase 0 — Foundation (months 1–2)

Objective: establish the partnership and load authoritative data. No passenger facing service in this phase.

Activity	Owner
Sign MOU; stand up joint programme office	Both
Bureau: approve stop, route, vehicle and operator registers	Bureau
Load Bureau-approved reference data into the platform	Team
Confirm fare policy as machine-readable versioned rules	Bureau
Procure and configure pilot infrastructure	Team
Sign key custody and rotation procedure	Bureau
Designate Bureau programme staff and auditors	Bureau
Security baseline review; confirm no development routes enabled	Both

Gate: MOU signed; data approved and loaded; key custody documented.

6.3 Phase 1 — Pilot (months 3–10)

Objective: prove revenue integrity and cash control on a real corridor with real money.

Scope: 6–10 routes on the central corridor; bus mode; cash, agent and one mobile payment method; passenger and staff apps; dashboard; complaints.

Activity	Notes
Integrate one live payment provider first	Telebirr recommended (largest reach). Adding both simultaneously doubles integration risk for no pilot benefit
Deploy staff handhelds with key enrolment	Enrolment is the key-lifecycle event
Train inspectors, conductors, agents, supervisors	§7.5
Begin shift and cash reconciliation	The core revenue control
Enable fare dual-control workflow	Bureau finance + bureau admin
Instrument baseline metrics	Must be measured before enforcement changes anything
Weekly variance review	The primary operational rhythm
Publish assurance dashboard to Bureau	Read-only, role-scoped

6.3.1 Staged enforcement

Enforcement intensity should **ramp**, not switch on. A sudden enforcement regime on a corridor where passengers have not adopted the app would generate grievances without revenue benefit.

Stage	Period	Approach
Observe	Months 3–5	Validate tickets where passengers voluntarily use them. Measure evasion baseline. No penalties.
Advise	Months 6–7	Officers explain and encourage. Penalties only for repeat, deliberate fraud.
Enforce	Months 8–10	Full enforcement, applied uniformly and audited

This sequencing should be agreed with the Bureau **before** the pilot starts, not negotiated once passengers are already affected.

6.3.2 Baseline first

We insist on measuring evasion and variance **before** enforcement begins. Without a baseline the Bureau cannot demonstrate improvement, cannot tune thresholds, and will be unable to distinguish a genuine drop in fraud from a change in enforcement intensity. This is the single most common failure in transport digitisation programmes, and it is cheap to avoid.

Gate: cash variance measured and trending; fare compliance measured; no unresolved severity-1 incidents; passenger adoption above the threshold agreed at Phase 0.

6.4 Phase 2 — Expansion (months 11–18)

Extend to additional operators and the wider central network. Introduce public open-data publication, Amharic interface localisation, and the refund workflow. Exit criteria from Phase 1 must be met in full — expansion does not begin to rescue a failing pilot.

6.5 Phase 3 — Integration (months 19–26)

Connect the AACRA ITS feed for live vehicle position and real running times; implement operator settlement runs and dual-controlled refunds. At this point Addis One begins to *consume* the ITS investment rather than merely complementing it.

6.6 Phase 4 — Citywide (months 27–50)

Full operator and route coverage; taxi and rail modes; open-data portal; Amharic first-language support; progressive transfer of operational ownership to Bureau staff under a documented exit protocol.

Part VII — Cost, Resources and Capacity

All figures are indicative planning estimates, not quotations. They are expressed in ETB and assume the pilot corridor scope in §1.6. Costs depend materially on Bureau-provided infrastructure, device procurement route, payment provider commercial terms, and staffing levels — all open questions at submission. We recommend the Bureau treat these as the basis for a budget request, to be refined jointly in Phase 0.

7.1 Phase 0 + Phase 1 (pilot, 10 months)

Line item	Estimate (ETB)	Notes
Software development	18.0M – 26.0M	One live payment provider, offline reconciliation, refunds, Amharic localisation
Infrastructure (pilot)	4.0M – 7.0M	Server, storage, backup, TLS, monitoring. Lower if the Bureau hosts
Devices	6.0M – 11.0M	Inspector/driver handhelds, agent phones. Largely offset if operators supply their own
SMS / OTP	0.9M – 1.8M	Volume-dependent
Payment provider fees	Variable	Commercial terms with Telebirr / CBE Birr; per-transaction
Network / data	1.2M – 2.4M	Staff app connectivity
Training & change management	2.5M – 4.5M	\$7.5
Security review / penetration test	1.5M – 3.0M	Not optional before real money
Programme management	2.5M – 4.5M	Joint Bureau–Team office
Contingency (~12%)	4.3M – 6.8M	
TOTAL (Phases 0–1)	41.0M – 67.0M	

7.1.1 Proposed cost sharing

The Bureau's cash requirement is a **minority** of total programme cost:

Source	Share	Mechanism
Team investment	40–50%	Software, engineering, programme management
Bureau	15–25%	Programme staff time, office, Bureau-side coordination, any device top-up
Operators	20–30%	In-kind devices, staff time, connectivity for their own fleets
Development partners	10–20%	Concessional finance or grant, if the Bureau wishes to pursue

Indicative Bureau cash contribution for Phases 0–1: ETB 8–16 million.

We recommend the Bureau not be asked to fund the software build in the pilot. The Bureau's scarce resource is not budget but **institutional attention** — staff time, policy decisions and enforcement legitimacy. Asking for cash *and* those would be asking for the same thing twice.

7.2 Phases 2–4 (citywide, 5 years)

Phase	Estimate (ETB)	Driver
Phase 2 — Expansion	70M – 110M	Second operators, wider network, localisation, refunds, open data
Phase 3 — ITS integration & settlement	85M – 130M	ITS feed work, settlement engine, real-time
Phase 4 — Citywide	225M – 380M	Full device fleet, all modes, long-term operations
TOTAL (Phases 2–4)	380M – 620M	

7.2.1 Recurring operational cost

Roughly **6–9% of capital cost per year** for hosting, support, security patching and key management. This must be budgeted from day one; systems that cannot be afforded to maintain become unmaintained, and an unmaintained revenue system is a liability.

7.3 Return model — and its honest limits

7.3.1 The mechanism

Source	Mechanism
Cash leakage recovery	Shift float + declared cash + variance makes leakage measurable; measurable leakage is recoverable
Fare dispute resolution	Disputes settled from a stored fare calculation rather than argument — lower administrative cost, fewer escalations
Enforcement efficiency	Targeting inspection at observed evasion hotspots rather than uniformly
Planning data	Origin–destination and load data for network planning — currently unavailable to the Bureau
Service quality	Structured complaint data directs operator accountability

7.3.2 Why we give no percentage

We have **not** run this system with real money, and we do not know the Bureau's current leakage rate. Any specific return percentage would be fabricated. Note that the Bureau also does not currently know its own leakage — which is precisely the problem this system solves.

The pilot's primary deliverable is a **defensible revenue baseline for the Bureau**, which no amount of desk analysis can substitute for.

7.4 Team and capacity

Role	Indicative FTE	Phases
Technical lead / architect	1	0–4
Backend engineers	2–3	0–3
Mobile engineers (Flutter)	1–2	0–3
DevOps / platform	1	0–4
Payment integration specialist	1	1
QA engineer	1	0–3
Security engineer	0.5	0–4

Role	Indicative FTE	Phases
Data analyst	1	1–2
Training lead	1	0–2
Programme manager	1	0–4
Peak total	~11–13	

7.5 Training and change management

Training is a **deliverable with an acceptance criterion**, not a line item.

Audience	Content	Duration
Inspectors / ticket officers	Scanning, outcome vocabulary, what each refusal means, device care	1 day
Conductors / agents	Shift open/close, cash declaration, variance, assisted sale	1 day
Supervisors	Variance review, dispute workflow, escalation	0.5 day
Bureau finance	Fare authoring and approval, dual control, settlement	1 day
Bureau administrators	Network reference data, staff roles, dashboard	2 days
Bureau IT (2–3 staff)	Deployment, backup/restore, key custody, log monitoring, handover	5 days

The last row is the programme's insurance policy: **at least two Bureau IT staff must be able to operate the platform independently** before Phase 4 begins.

7.5.1 Change-management realities

The honest risk in this programme is not technical. It is that **frontline staff resent the system** if it is perceived as surveillance or as a pretext for sanctioning. Two mitigations are built into the design:

- Scan history is the officer's **own**, not a city-wide who-inspected-whom feed (§4.5.7) — a deliberate privacy protection for staff.
- Staged enforcement (§6.3.1) means officers are not asked to enforce an unfamiliar rule against the public before the public has adopted the system.

We recommend the Bureau communicate these protections explicitly at launch. Staff trust is worth more to this programme than any feature.

Part VIII — Success Metrics

Metrics are grouped by the objective they serve. Each has a **baseline measured in Phase 1 before enforcement** and a target agreed at the Phase 0 gate. We do not propose targets here, because setting them without a baseline is how programmes end up claiming success they did not achieve.

8.1 Revenue integrity (O1, O2)

Metric	Definition	Source
Shift declaration rate	Shifts closed with a declared cash figure ÷ total shifts	Shift records
Variance distribution	Mean, median and standard deviation of variance by operator	Shift records
Variance trend	Monthly movement in variance rate	Shift records
Fare calculation reproducibility	Share of tickets whose fare can be reproduced from stored inputs	Fare calculations
Fallback fare rate	Share of journeys priced by fallback rather than a matched rule	Fare engine flag
Unreconciled revenue	Revenue in terminal payment state not reflected in a closed shift	Payment ↔ shift

The fallback fare rate deserves attention — it is the system's own internal alarm that fare policy has a coverage gap.

8.2 Compliance and enforcement (O3)

Metric	Definition	Source
Validation rate	Validations ÷ tickets issued, per route/day	Validation records
Outcome distribution	Share of each of the 11 outcomes	Validation records
Repeat forgery attempts	Distinct devices/staff linked to <code>INVALID_SIGNATURE</code>	Validation + audit
Double-scan rate	Second scans of already-validated tickets	Validation records
Overcharge complaint rate	<code>OVERCHARGE</code> complaints per 10,000 tickets	Complaints

8.3 Service quality (O4)

Metric	Definition
Complaint volume and trend per route/operator	
Category mix (share of each of 8 categories)	
Geolocation coverage	
Resolution rate and median time to resolution	
Repeat-complaint rate for the same operator	

8.4 Data quality (O5)

Metric	Definition
Stops/routes/vehicles with complete attributes	

Metric	Definition
Fare rule coverage: zone-pairs with no active rule	
Orphan reference data (routes with no valid stops)	
ITS feed freshness, where integrated	

8.5 Adoption and equity

Metric	Definition
Active unique users, and transactions per active user	
Share of transactions by method (cash vs mobile)	
Concession uptake — are discounted groups actually using the system?	
Coverage of outer sub-cities vs the central corridor	

We flag the equity metric deliberately. A ticketing system that only serves smartphone-owning commuters in the centre would entrench exclusion while reporting success. Concession uptake and sub-city coverage should be reported to the Bureau as first-class indicators, not afterthoughts.

8.6 System health

Metric	Threshold expectation
API availability	≥99.5% excluding planned maintenance
Payment success rate	Baseline in pilot; investigated if it drops
P95 API latency	Sub-second for reads; sub-3s for purchase
Database growth and index usage	Reviewed monthly
Security alerts	Zero unresolved severity-1
Backup/restore drill	Passed at least once per quarter

Part IX — Risk Register

Risks are scored **likelihood × impact** (H/M/L). We present the top risks honestly, including those to **this proposal**.

#	Risk	L	I	Mitigation	Owner
R1	Low passenger adoption — cash culture persists, smartphone/data cost barrier	H	H	Keep cash and agent paths first-class; agent-assisted sales; staged enforcement; measure adoption weekly from month 1	Both
R2	Frontline staff resistance — perceived as surveillance or sanction tool	H	H	Officer-private scan history; staged enforcement; involve unions early; communicate protections explicitly	Bureau
R3	Payment provider integration delay	M	H	Integrate one provider first; commercial terms in Phase 0; cash path independent of providers	Team
R4	Enforcement backlash without baseline	M	H	Measure before enforcing (§6.3.2); uniform audited application	Both
R5	Reference data quality — Bureau data incomplete or contested	M	H	Bureau signs off data as authoritative; data-quality dashboard from day one	Bureau
R6	Connectivity gaps in outer sub-cities	H	M	Offline validation; local queue; store-and-forward	Team
R7	Key compromise — signing key stolen	L	H	Keys in Bureau custody (HSM where available); documented rotation; <code>keyId</code> enables non-breaking rotation; revocation	Bureau
R8	Insider abuse — staff collusion or credential misuse	M	H	Hash-chained audit; dual control; operator boundary; periodic audit reviews	Bureau
R9	Vendor dependency — Team unavailable	M	M	Source in Bureau environment; handover and escrow; Bureau IT trained from Phase 1	Both
R10	Privacy incident — passenger data exposure	L	H	Data minimisation by design; role scoping; retention policy; penetration test	Both
R11	Fare policy disputes during changeover	M	M	Versioned fares; dual control; published effective dates; grandfathering rules	Bureau
R12	Over-reliance on mock provider assumption	L	H	Explicit acceptance testing against live sandbox before go-live	Team
R13	Budget not sustained for operations	M	H	Recurring cost budgeted from Phase 0, not deferred	Bureau
R14	Equity failure — system serves only the connected centre	M	H	Concession uptake and sub-city coverage as first-class metrics; agent and cash paths retained	Both

9.1 Risks to this proposal specifically

We state these plainly, because a proposal that lists only risks to others is not being candid:

1. **We have not moved live money.** The orchestration guarantees are tested, but real-world provider behaviour will surprise us. R3 and R12 are the mitigations.
2. **The journey planner is deliberately simple.** It will not produce the theoretically optimal itinerary in every case. This is a deliberate trade for explainability, but the Bureau should confirm it accepts that trade.
3. **Amharic UI is not built.** Interface strings are English. For Phase 1 this is acceptable for staff and supervisors, and **less acceptable for direct citizen use**. Localisation is Phase 2 work and should be funded before broad public rollout.
4. **Cost estimates have wide ranges.** They narrow only after Phase 0 scoping.

5. **Adoption is the dominant risk** and it is behavioural, not technical. No engineering fixes a passenger who does not want to change how they board a bus. This is why the pilot exists.

Part X — The Request

10.1 What we ask the Bureau to approve

#	Request	Decision needed
1	Authorise a pilot on 6–10 central-corridor routes over 10 months	Bureau leadership
2	Establish a joint programme office with named Bureau and Team staff	Bureau leadership
3	Designate Bureau-approvable reference data — stops, routes, vehicles, operators, fare policy	Bureau + operators
4	Grant Bureau custody of signing keys and approve the rotation procedure	Bureau security
5	Nominate Bureau auditors with read access to the audit chain and revenue reporting	Bureau
6	Confirm enforcement staging before pilot launch	Bureau
7	Authorise payment-provider engagement (Telebirr first)	Bureau + Team

10.2 What we commit to

1. **Full transparency on capability** — §4.12 states plainly what is built, partial and not started.
2. **Bureau verification, not Bureau trust** — §4.12.5 lists the demonstrations we will perform on request, including attempts to break the system.
3. **Bureau-owned data and keys** — §5.3.
4. **Bureau IT capability before citywide** — at least two staff independently operating the platform (§7.5).
5. **Measured baselines before conclusions** — no revenue claims without data.
6. **No lock-in** — standard formats, documented exit, escrow.

10.3 Immediate next steps

We propose a **two-hour technical and programmatic briefing** at the Bureau's convenience, covering:

- Live demonstration of the fare dispute workflow — the most persuasive artefact in this proposal
- Demonstration of the four invariant guarantees
- Architecture and security review
- Cost and phasing discussion
- Agreement on pilot corridor and enforcement staging

We will bring the running system. We would rather be judged on a demonstration than on this document.

10.4 Contact

Programme	Addis One — Digital Public Transport Modernisation
Documentation	docs/architecture.md , docs/adr/0001-modular-monolith.md , docs/adr/0002-ed25519-qr-signing.md
Evidence available on request	Source repository, 141-test suite (7 suites, all passing), live demonstration environment, architecture review

Appendix A — Glossary

Term	Meaning
Fils	Integer minor unit of the birr. 1 ETB = 100 fils. All money is stored in fils
Idempotency key	Client-generated token ensuring a repeated request performs one charge, not many
Credential	The signed data in a ticket QR: identifiers, timestamps, nonce, signature — no commercial data
Validation	An inspector's scan determining whether a passenger may board
Variance	Difference between expected cash (float + sales) and declared cash at shift close
Freshness	How current a reading is: <code>REAL_TIME</code> , <code>ESTIMATED</code> , <code>SCHEDULED</code> , <code>STALE</code> , <code>UNKNOWN</code>
Operator boundary	Staff may act only on their own operator's resources unless holding a bureau or finance role
Dual control	Requiring two separate people to complete one sensitive action
Fallback fare	A flagged default price used when no fare rule matches; an alarm, not a normal path

Appendix B — Architecture Decision Records

ADR	Decision	Status
0001	Modular monolith, not microservices — because ticket issuance and payment confirmation must commit atomically, and network services would add partial-failure modes to a system whose correctness depends on that guarantee	Accepted
0002	Ed25519 for QR credential signatures — driven by the offline-verification constraint: 32-byte public keys distribute to thousands of handhelds, 64-byte signatures keep QR codes scannable, and deterministic signatures remove a class of replay questions	Accepted

ADR 0001's revisit condition: extraction into separate services should be considered when a module needs independent scaling (fleet GPS is the first candidate), a different release cadence, a team boundary requiring hard isolation, or a genuinely distributed deployment target.

Appendix C — Technical evidence available for Bureau review

Item	Location / detail
Architecture document	<code>docs/architecture.md</code>
Modular monolith rationale	<code>docs/adr/0001-modular-monolith.md</code>
Ed25519 rationale	<code>docs/adr/0002-ed25519-qr-signing.md</code>
Canonical data model	<code>backend/prisma/schema.prisma</code> (~40 models)
Fare engine + tests	<code>backend/src/modules/fares/fare-engine.ts</code> , <code>.spec.ts</code> (21 tests)
Ticket state machine + tests	<code>backend/src/modules/ticketing/ticket-state-machine.ts</code> , <code>.spec.ts</code> (24 tests)
Payment orchestrator + tests	<code>backend/src/modules/payments/payment-orchestrator.ts</code> , <code>.spec.ts</code> (23 tests)
Credential signer + tests	<code>backend/src/modules/ticketing/credential-signer.ts</code> , <code>.spec.ts</code> (18 tests)
Authorisation policy + tests	<code>backend/src/modules/auth/policy.ts</code> , <code>staff-session.spec.ts</code> (25 tests)
Dashboard query + tests	<code>backend/src/modules/dashboard/dashboard-query.ts</code> , <code>.spec.ts</code> (20 tests)

Item	Location / detail
Config validation + tests	<code>backend/src/config/validate-env.ts</code> , <code>.spec.ts</code> (11 tests)
Local infrastructure	<code>infra/docker-compose.yml</code> (PostgreSQL, Redis, TimescaleDB)
Real reference data	<code>backend/prisma/seed.ts</code> — 15 stops, 6 routes, 2 operators, versioned fare rules

Verified at submission: 7 test suites, 141 tests, all passing.

Suite	Tests	Covers
<code>ticket-state-machine.spec.ts</code>	24	I1 — single edge into <code>TICKET_ISSUED</code> , terminal immutability
<code>staff-session.spec.ts</code>	25	Role lists, operator boundary, city-wide scoping
<code>payment-orchestrator.spec.ts</code>	23	I2 idempotency, terminal states, provider contract
<code>fare-engine.spec.ts</code>	21	Specificity ranking, versioned rules, floors, concessions, transfers
<code>dashboard-query.spec.ts</code>	20	Date range and pagination resolution
<code>credential-signer.spec.ts</code>	18	Ed25519 sign/verify, canonical serialisation, key selection
<code>validate-env.spec.ts</code>	11	Configuration validation and password redaction

End of proposal. Prepared October 2026. All technical claims verifiable against the inspected system; all cost figures are indicative planning estimates requiring Phase 0 refinement.